6章
イベントと通知の管理

イベント駆動型データアーキテクチャは新しいパラダイムです。イベント駆動型データアーキテクチャによって、リアルタイムでデータ駆動型のエクスペリエンスへとビジネスを変革することができます。ただし、強力な反面、複雑すぎます。多くのサービス、追加コンポーネントを必要とし、フレームワークも必要です。また、あらゆる種類の課題に対するソリューションでもありません。どのような種類のイベント駆動型通信を導入する場合でも、これらの懸念事項について検討する価値があります。

この章では、イベントと通知の管理に重点を当てます。これらは複雑な領域です。API管理やデータプロダクトアーキテクチャ設計と重なる部分があるからです。非同期通信、イベント駆動型アーキテクチャ、最新のクラウド技術、一貫性モデル、イベント種別などについて見ていきます。この章を読み終える頃には、イベント駆動型アーキテクチャが組織にもたらすものと、それに伴うトレードオフについて、十分理解されていることと思います。

いくつか先に言っておきたいことがあります。まず、すべてのデータ担当者がイベント駆動型アーキテクチャに精通しているわけではありません。ここでは基本的なことを説明し、データ管理という観点から見たイベント流通に絞って説明します。次に、イベント駆動型アーキテクチャは複雑になる恐れがあります。このトピックについて完全に説明すると、これだけで本書が埋まってしまうかもしれません[†1]。ここでは、重要な部分に絞って、参考例を用いて、ガバナンスモデルと留意すべき重要な点を説明します。

[†1] イベント駆動アーキテクチャについてもっと知りたいなら、Ben Stopford著“Designing Event-Driven Systems”（O'Reilly）というすばらしい本があります。

6.1　イベントの紹介

アーキテクチャとガバナンスについて説明する前に、イベントデータについて、それがどのようなものなのか、そしてなぜ非同期通信が重要なのかについて紹介したいと思います。イベントデータとは、ある時点で発生した変化（**イベント**）に関する情報です。イベントデータは、競争上の優位性をもたらします。より迅速に結果を提供することでインサイトはより適切なものになり、より速い対応はより高い顧客満足度につながります。数分、数時間待つことなく、即座に対応することができます。不正行為を、不正行為が行われた瞬間に発見することができます。Webサイトへの訪問者がどこから来ているのかを確認し、Webサイトに滞在している間にカスタマーエクスペリエンスを改善することができます。イベントストリームとして配信されるデータは、組織にとって非常に大きな価値があります。そのソースは、IoT（Internet of Things、モノのインターネット）デバイスや、スマートフォンのクリック、ゲーム内のプレイヤーの行動、ソーシャルネットワーク、従来型の運用システムにおける変化など、非常に多岐にわたります。

オフラインバッチやAPIリクエストに（リアルタイムで）応答して配信されるデータとは異なり、イベントデータは、イベントとのやり取りやイベントへの反応に重点を置いています。アプリケーションとシステムを**非同期で**接続することが目的です。イベントデータは高いスループットを実現することを重視し、ステートレスにも、ステートフルにもなります。ある特定の時点で発生した変化に関する情報を交換し、アプリケーションの状態をある場所から別の場所に移動することができます。

イベント駆動型アーキテクチャでは、多くの場合、イベントストリーミングプラットフォームを使用します。イベントストリーミングプラットフォームは、データがプラットフォーム内を流れる間に、受信したイベントを処理します。これらのソフトウェアプラットフォームはさまざまな「顔」を持ち、さまざまな機能をサポートします。例えば、以下のようなことができます。

- リアルタイムのETLおよびデータ統合プロセスを実行することで、イベントのストリームを操作します。
- イベントのウィンドウ分析を実行し、パターンや相関関係を検出することで、リアルタイム分析を行います。
- リアルタイムでデータを保存し、ストレージ間で移動します。
- 機能を起動し、ほかのシステムに更新情報を発行します。

データプロダクトの保存方法やAPIの使用方法と重なる部分があります。このため、これらのイベント駆動型パターンでは、非常に高度なデータ管理が求められます。例えば、1つのイベントが多数のメッセージや多くの異なるサービスへの呼び出しをトリガーすることがあります。データ管理の観点からは、どの返信がどのメッセージに対するものかを追跡したくなるでしょう。

6.1.1　通知と状態転送

イベントにはさまざまな形があり、データ流通やデータ管理と重なるものもあります。イベントを流通させる方法の1つに、**イベントによる状態転送**と呼ばれるものがあります。この方法では、イベント自体が変更に関連する状態をすべて運ぶので、コンシューマは必要な情報をすべて持つことになります。イベントを通知された後に、別途リクエストを送信して状態を得る必要はありません。コンシューマは、これらのイベントを使用して、データのローカルコピーを構築することもできます。**例6-1**は、「add customer」（顧客の追加）イベントの例です。イベントは、古いデータと新しいデータの両方を含むような形にすることもできます。例えば、「update customer」（顧客の更新）イベントの場合、コンシューマは変更された情報を正確に確認することができます。

例6-1　顧客の変更に関連する全状態を伝えるイベントの例

```
{
    "id": "321125215",
     "firstName": "Adolfo",
     "lastName": "Prosacco",
     "gender": "male",
     "birthday": "1997-05-01",
     "address":
     {
         "streetAddress": "17795 Louisiana 182",
         "city": "Baldwin",
         "state": "Louisiana",
         "postalCode": "70514",
         "country": "United States"
     },
     "phoneNumber":
     [
         {
           "type": "home",
           "number": "(813) 872-1350"
```

```
            },
            {
              "type": "mobile",
              "number": "(202) 555-0154"
            }
        ]
  }
```

別の方法として、状態をすべて通知せずに、イベントに関する通知だけを送信する方法もあります。この場合、イベントには、変更された状態への参照という最低限の情報のみが含まれます。**例6-2**は単純化しすぎましたが、このようなイベントを受け取ったコンシューマは、プロデューサーに対して折り返し呼び出しを行い、変更に関する詳細な情報を取得しなければなりません。

例6-2　顧客変更の通知を表すイベントの例

```
  {
      "id": "321125215"
  }
```

通知を行う意図は、コンシューマが反応して何らかのアクションを起こすように促すことです。このような種類のイベントは、データ管理の観点からはあまり面白いものではないかもしれません。ただし、企業統合の観点から、クロスドメインのイベントが従うべき標準を設定したいことは確かです。

6.1.2　非同期通信モデル

リクエスト／レスポンス型通信とイベント駆動型通信の決定的な違いは、データ伝送における会話の方向が逆転していることです。イベントでは、話すのではなく、聞くことになります。ほとんどのユーザーがこの考え方に慣れていないので、複雑さを感じる点でもあります。多くの人々は、何かを実行に移すとき、即座に反応が返ってくることを期待します。例えば、リンクをクリックしたら、すぐに新しいページが表示されることを期待します。電話を取って話し始めれば、相手の声が聞こえてくることを期待します。要求があり、それに応えるという同期型の通信を、自然なことだと感じています。

一般的に、ストリーミングクライアントはプラットフォームとの接続を開いて、コンテンツがプッシュ（配信）されるのを待ちます。この通信モデルにおける接続は、データを交換できる期間が限られています。この期間を過ぎたら、すべての接続を閉じて、リソースを解放する必要があります。接続の開閉を繰り返すよりも効率的な通信形態は、接続をできるだけ長く開いておくことです。この手法は**ロングポーリング**（https://oreil.ly/7Bwfw）と呼ばれています。

一方、反応型の通信は、考え方が異なります。誰かが連絡してくるのを待ち、それからアクションを起こすのです。アプリケーションや統合プラットフォームは、イベントを使って同じことをします。イベントやメッセージをメッセージキューに送り、ほかのシステムがそれを取得するのを待つのです。このような疎結合により、ほかのパーティがすぐに反応するのを待つという依存関係がなくなります。このように非同期通信は、待つことができる状況においては、アーキテクチャを、よりレジリエンス（回復力）のあるものにしてくれます。

次に、イベント駆動型アーキテクチャがどのようなものかを説明し、イベントを生成、配信、消費するためのさまざまなパターンを見ていきます。

6.2　最新のイベント駆動型アーキテクチャとは

イベントがアプリケーション間で流通する、イベント駆動型アーキテクチャ（EDA）は、要件によって実装方法が異なります[†2]。さらに、多くのプラットフォームやサービスは、それぞれ異なる機能セットを提供しています。特定の機能に特化しているサービスもあれば、可能な限り包括的であろうとするサービスもあります。いくつかのバリエーションについて説明しましょう。イベント駆動型のメッセージングとデータ流通がどのように機能するかを理解する上で役立つでしょう。

[†2] Clemens Vasters氏は、非同期メッセージングとイベントをテーマに、たくさんのプレゼンテーション、例、リンク、ドキュメントを集めたすばらしいサイト（https://oreil.ly/UOM8Y）を運営しています。

6.2.1　メッセージキュー

アプリケーション間でイベントを共有するための有名な手法の1つに、メッセージキューを使用するものがあります[†3]。メッセージキューは、その名前からわかるように、イベントやメッセージが処理されるまでの待ち行列のことです。メッセージキューは、長年にわたりメッセージ処理に使用されており、さまざまなコミュニケーションスタイルを容易に実現できます。一般的な例は**ポイントツーポイントの消費モデル**です。**パブリッシャー**がキューにメッセージを送り、**サブスクライバー**がキューからメッセージを取り出します。メッセージキューは、メッセージブローカーとも呼ばれ、高速に動作します。メッセージが一度だけ処理されることを保証するために、コンシューマが受け取ったことを確認した後に、メッセージは削除されたりフラグが立てられたりします。

[†3] メッセージキューはメディエータトポロジーモデルを実装するために使用できます。これは、イベント処理を管理するために中央で調整を行うアーキテクチャ設計です。

メッセージ待ち行列は、通常、トランザクション処理あるいは整合性が要求されるイベント処理に使用されます。例えば、各メッセージが一度だけ、あるいは特定の順序（通常は先入れ先出し）で処理されることが重要な場合や、配送保証や未配信メッセージを格納するためのデッドレターキューのような機能が必要な場合などです。

6.2.2　イベントブローカー

アプリケーション間でイベントを共有する別の方法として、**イベントブローカー**を使う方法があります。イベントブローカーは、アプリケーション間の直接通信を実現する軽量なミドルウェアです。アプリケーションが異なる言語を使って記述されていても、異なるプロトコルを使って実装されていても問題ありません。イベントブローカーはアプリケーション間の仲介を行い、一般的に、

イベントを検証し、保存し、ルーティングし、宛先に直接配信します。図6-1に示すように、イベントブローカーにはさまざまな種類があります。

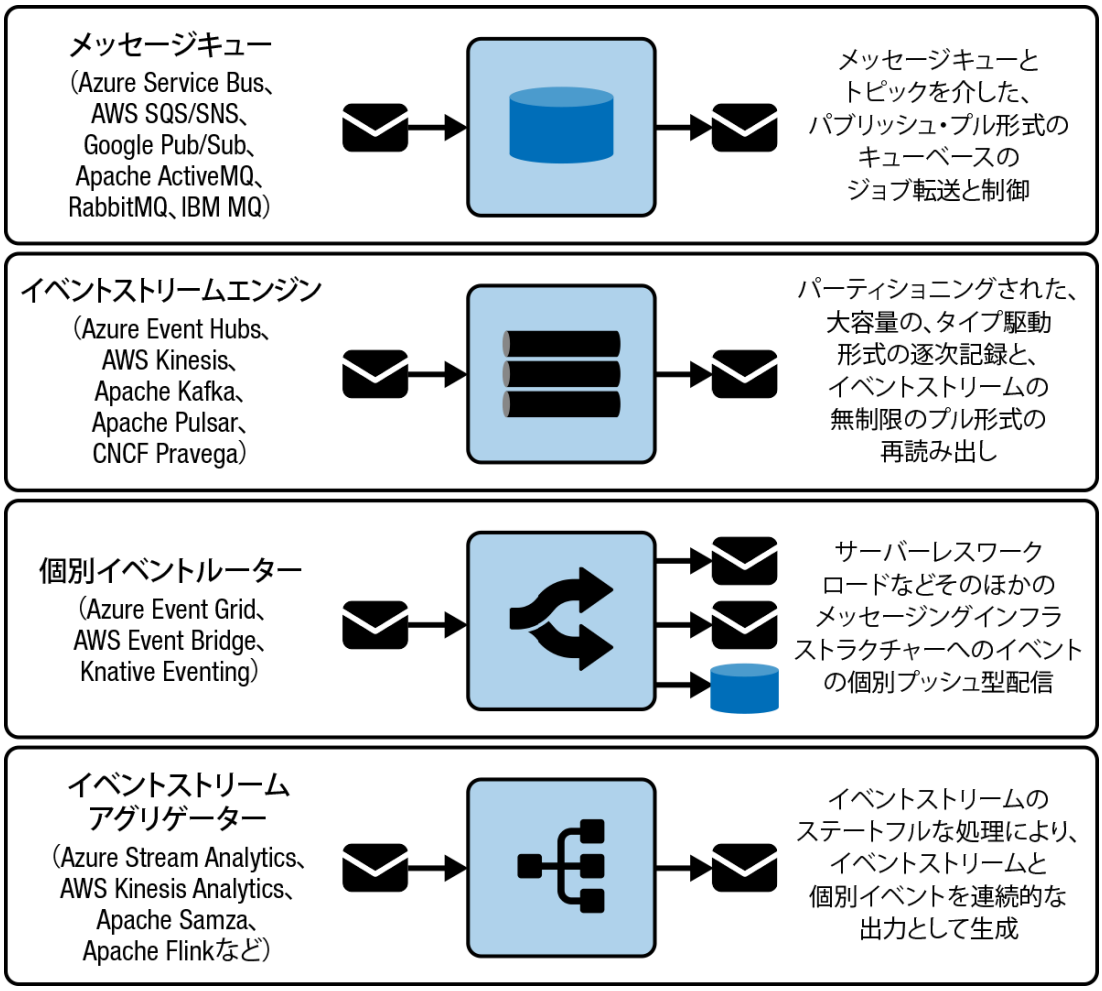

図6-1　さまざまなイベント配信パターン

図6-1で言及したようなイベント駆動型プラットフォームやサービスは、ファイルをファイルシステム内のフォルダで整理するのと同じように、イベントをグループ化して整理します。イベントがグループ化される構造には、それぞれ異なる名前が使われます。例えば、Apache Kafkaは**トピック**という言葉を使います。Azure Event Hubsは**イベントハブ**という言葉を使います。Amazon Kinesisは**ストリーム**という言葉を使います。これらの言葉は、基本的にはすべて同じパターンを指しています。論理的に一緒になるべきイベントをグループ化するというパターンです。

ほとんどの種類のイベントブローカーは、アプリケーション間でイベントのデータパイプラインやストリームを直接提供します。この点が、メッセージキューとは異なります。単純にサービス間でイベントをルーティングするだけのものもあれば、通り過ぎるイベントを集約したり、学習したりするものもあります。イベントストリームルーターとは異なり、イベントストリームアグリゲーターはイベントの内容を調べます。イベントブローカーの3つ目の種類はイベントストリームエンジンです。分散ストレージ機能を使用して、データフローを取り込みます。信頼性が高く、メッセージ配信を確実にするために、エンジンによっては、消費アプリケーションが処理できるようになるまで、イベントを保存し、順序付けするメッセージキューを利用するものもあります。

6.2.3　イベント処理スタイル

EDAのトポロジーは、その複雑さやユースケースによって、イベント処理のスタイルが異なります。同一のアーキテクチャで複数のスタイルが組み合わされることもあります。

シンプルイベント処理（SEP、Simple evnet processing）は、最もシンプルな処理スタイルです。ある特定の状態の変化に対して、直接関連するイベントを処理します。過去のイベントが現在のイベントの処理に影響を与えることはあ

りません。複雑なワークフローは発生しません。実例としては、変化を検知することでイベントを送信するセンサーなどです。

イベントストリーム処理（ESP、Event stream processing）は、複数のイベント（メッセージ）を同時に処理します。SEPとの違いは、過去のイベントの状態が未来のイベントの処理方法に影響を与える場合があることです。実際の例は株式取引です。株価データのストリームを分析し、株を買うか売るかを決定します。

複合イベント処理（CEP、Complex event processing）は、イベント処理の中で最も複雑な形態です。より複雑なイベントが発生したかどうかを判断するために、パターンを見つけるための分析が行われるからです。対象となるイベントは、長期間にわたって評価が行われます。複数のソースから発生する場合もあります。例えば、複数のストリームを結合して1つのストリームを作成し、そのストリームに集約処理を施して、カウントや、平均、合計を計算することができます。また、スライディングウィンドウ関数やタンブリングウィンドウ関数など、高度なウィンドウ操作を行うこともできます。実例としては、リアルタイムの不正検知、インフラストラクチャーの監視とアラート、低遅延のダッシュボード更新などがあります。

サービス指向アーキテクチャとイベント駆動型アーキテクチャの比較

SOAとEDAはどう違うのでしょうか。あるいは、これらのアーキテクチャスタイルは同じものなのでしょうか。どちらのアーキテクチャもより高いアジリティの実現を目指し、サービス合意、サービスのディスカバラビリティ、サービスの再利用可能性、サービスの抽象化、サービスの複合化という設計原則を提唱しています。大きな違いは、EDAではデータの複製が許可されるため、ある程度のデータの不整合を許容しているのに対し、SOAではサービスの分離により不整合を回避している点です。SOAは疎結合の原則を提唱しています。各サービスがデータベースや、レガシーアプリケーション、APIなどほかの共有リソースへ依存することが制限されており、隔離されたエンティティとなります。また、SOAがコマンドに重点を置くのに対し、EDAはイベントに重点を置きます。

イベント駆動型アーキテクチャの実装は複雑です。メッセージキューやイベントブローカーを配置するだけではありません。完全なEDAには、イベントを作成、収集、変換、保存、消費するための多くのサービスやコンポーネントが必要です。イベント駆動型アーキテクチャがどのように機能するかを深く理解で

きるように、以降の項ではさまざまな領域を深く掘り下げていきます。まず、プロバイダー側について説明し、イベントがどのように生成されるかを学びます。次に、消費側について説明します。最後に、この2つを結びつけ、スケーラブルにイベントを管理する方法を学びます。

6.2.4　イベントプロデューサー

イベントプロバイダー（多くの場合、**イベントプロデューサー**または**パブリッシャー**と呼ばれます）側のソースシステムは、普通は、それ自体でデータをストリーミングしたり、イベントを生成したりしません。一般的には、データソースからデータを収集し、プロトコルを変換したり、メッセージを適切なフォーマットに変換したり、フィルタリング、集約、リッチ化などの統合を実行したりするために、統合コンポーネントを追加する必要があります。イベントの生成は、アプリケーションとデータベースの2つの異なるレベルで行われます。シナリオや選択肢はいくつか存在します。よく知られているものとそのベストプラクティスを見てみましょう。

6.2.4.1　アプリケーションが生成するイベント

ほとんどの場合、アプリケーションはイベントや通知を自動生成しません。そのため、イベントを生成するためには、（開発者である）読者自身がアプリケーションのアーキテクチャを修正する必要があります。状況やアプリケーションによって、これらの修正は簡単な場合もあれば複雑な場合もあります。いくつか例を挙げましょう。

イベントの生成や送信には、通常、クライアントライブラリを追加する必要があります。例えば、Expressで記述されたNode.jsアプリケーションにおいて、Kafkaにイベントを送信する場合、おそらくはKafkaJSクライアントライブラリを使用することになるでしょう。しかし、アプリケーションをAzure Event Hubsに接続する必要がある場合は、おそらく別のクライアントライブラリ、つまりJavaScript用のAzure Event Hubsクライアントライブラリを使うでしょう。まとめると、使用する言語、フレームワーク、ミドルウェアにはそれぞれ相互の依存性があるということです。

イベントを発生または収集するもう一つの方法は、サービス間のトラフィックを直接取り込むことです。最新型のアプリケーションやWebサービスは、フロントエンドや、ユーザーインタフェース、基盤となるアプリケーションコンポーネントを通じて、イベントが直接生成されるように設計されています。例え

ば、Webページの閲覧やショッピングカートに商品を追加するようなブラウザ内のイベントは、APIを介して直接受け取ることができます[†4]。

[†4] フロントエンドとユーザーインタフェースによって生成されるイベントは、通常、一方向（fire-and-forget）の仕組みです。メッセージの受信者は応答する必要がない場合が多いからです。

イベントは、アクセスログやサーバーログなどのログを解析することによっても生成できます。この場合、異常があればイベントが発生します。多くの場合、Node.js の例と同じように、イベント生成を行うにはコンポーネントを追加する必要があります。

APIなしで開発された複雑なアプリケーションやレガシーアプリケーションでは、イベントの受け取りが問題になることがあります。例えば、Visual Basic 6で書かれたアプリケーションを使用している場合、アプリケーションを最新化するか、統合用にデータベースを使用する必要があります。

6.2.4.2　データベースが生成するイベント

アプリケーションのデータベースに対する変更を読み出すことによってイベントを生成する方法は、APIやサービスの代わりとなる方法です[†5]。変更の読み出しは、ポーリング、トリガー、変更データキャプチャ（CDC、change data capture）を使って行えます。このイベント配信モデルは、イベントによる状態転送と密接に関係しています。アプリケーションがイベントを生成する場合と同じように、より多くのコンポーネントと追加の作業が必要になるかもしれません。

[†5] この形式の状態は、**リソース状態**とも呼ばれ、ある時点におけるサーバー上のリソースの現在の状態です。エンジニアによっては、クライアントに存在するアプリケーション状態と、サーバーに存在するリソース状態を区別する人もいます。

最初に紹介する方法は、データベースコネクタを使用したポーリングベースの方法（https://oreil.ly/3lxkV）です。データベースのテーブルは、（一般的には、Open Database ConnectivityやJava Database Connectivityを使って）クエリを繰り返し実行することで、メッセージキューやイベントストリームにマッピングされます。テーブルに新しいレコードが挿入されたり、行が更新されたりするたびに、そのレコードがテーブルから取得され、メッセージキューまたはイベントストリームに転送されます。ポーリングベースの方法は比較的簡単に構築できますが、絶えず問い合わせを行うため、データベースへの負荷が高くなる場合があります。ポーリングのもう1つの欠点は、アプリケーションとデータベースの設計を変更する必要があることです。例えば、データベースから取得しなければならない行を検出するために、`updated_at`カラムを追加するなどの準備が必要になるかもしれません。

ポーリングとは別に、トリガーベースの方法でイベントを生成することも可能です。**トリガー**は、ストアドプロシージャやデータベースフックなどのプロシージャコードの一部であり、指定されたイベントが発生した時のみ実行されます。例えば、あるテーブルにデータが追加されるたびに実行されるトリガーを用意します。このトリガーは、レコードをイベントテーブルや変更フィードに移動させたり、 関数を呼び出してデータを別のコンポーネントに送信したりします。トリガーの問題点は、データベースの負荷が高い時に処理が遅くなることです。もう1つの欠点として、すべてのデータベースがトリガーをサポートしているわけではないということがあります。

状態を取り込むためのもう一つの方法は、CDCを使用することです[†6]。これは、特殊なデータベースツールを使用し、データベースのトランザクションまたは監査ログからすべての変更を読み出すことで機能します。CDCベースのツールは、データベースに直接クエリを行わないため、 データベースの負荷は少なくなりますが、多くの場合、ほかの商用コンポーネントやサードパーティコンポーネントが必要になります。

[†6] Azure SQLとDebeziumでCDCを使用する例は、Microsoftのドキュメント（https://oreil.ly/qz4tE）を参照してください。

アプリケーションデータベースから直接データを取り込むのは簡単ではありません。多くのアプリケーションは運用ワークロード用に設計されています。こ

れらのアプリケーションは、整合性を保証するために、複雑で高度に正規化されたデータ構造を使っています。このような場合、変換や抽象化が必要になります。このような場合に有効なベストプラクティスは、ビューを使用したり、データの処理済みのバージョンを別の（一時的な）テーブルに書き込んだりすることです。そして、この場所からデータを読み出します。この方法を使うと、後からリアルタイムで複雑なデータ操作を行うことが避けられます。

これらのさまざまな例からどのような結論が導き出せるでしょうか。それぞれのアプリケーションには独自の特性があり、カスタムソリューションやサードパーティのコンポーネントが必要になる場合があります。したがって、データ管理については、アプリケーション統合チームと開発者が、中央プラットフォームチームと緊密に連携して、標準とベストプラクティスを策定することが重要です。例えば、コネクタが利用できない場合、中央プラットフォームチームはコネクタを構築し、それをほかのドメインチームに提供する必要があります。

6.2.5　イベントコンシューマ

では、イベントコンシューマ側に切り替えてみましょう。イベントコンシューマ側では、ユースケースの要件によってバリエーションが決まります。いくつかのシナリオを見ていき、可能性について議論してみましょう。消費パターンは、ほかのドメインの性能に直接影響するため、重要です。この項では、（決してすべてを網羅しているわけではありませんが）いくつかのケースについて説明します。

メッセージキューやメッセージブローカーを利用する一番の理由（そして最もシンプルなユースケース）は、通信に中間のレイヤーを利用することで、プロバイダーとイベントコンシューマを分離することです。プロデューサーはメッセージを発行し、コンシューマは消費ワークロード用にメッセージを消費します。実際の例としては、どの顧客がどのWebサイトを訪問したかを把握するために、Webサイトの利用状況を追跡し、マーケティングアプリケーションにイベントを発行するようなユースケースがあります。

単純なバリエーションとして、複数のコンシューマが、1つのプロデューサーから同じイベントストリームを消費する場合があります。この場合、それぞれのコンシューマは自分自身でメッセージオフセット（どこまでメッセージを消費したか）を管理します。このパターンは**ファンアウト交換**と呼ばれます。実

際の例としては、顧客に関するすべての変更を複数の下流アプリケーションに配信するような顧客管理システムがあります。

より複雑な状況としては、運用システムにおいて、クラウドの機能を利用することで内部アーキテクチャを強化する場合が挙げられます。この場合、機能ベースのサービスがクラウドに配備され、イベントを使って呼び出されます。この機能はメッセージを処理し、結果を保存して（おそらく、ほかのサービスを呼び出すことで結果をより詳細にし）、運用システムに送り返します。このパターンは**ストラングラーパターン**とも呼ばれます。この例での機能（アプリケーションコンポーネント）は、（サーバーレスの）マイクロサービスとなるでしょう。

高度なシナリオとしては、ストリーム処理エンジンや分析プラットフォームを使用して、複数のイベントプロバイダーから集約されたイベントストリームを分析し、パターンを探す場合が考えられます。ある特定の時間ウィンドウ内で特定のイベントが発生したり、データセットが一致したりすると、別のイベントがトリガーされます（おそらく、自動化されたアクションが開始されたり、ほかのシステムにこのイベントが発生したことが通知されたりします）。前述したように、データベースに格納する前にデータを照会・分析するこのパターンは、**複合イベント処理**と呼ばれています。多くの先進的なストリーミングプラットフォームでは、ウィンドウ機能（タンブリング、ホッピング、スライディング、セッションなど）をサポートすることで、分析処理を行うことができます[†7]。

[†7] Microsoftは、Azure Stream Analyticsのウィンドウ機能を詳しく紹介しています（https://oreil.ly/BV-gN）。

イベントストリーム処理と複合イベント処理の比較

イベントストリーム処理と複合イベント処理の違いについては、Srinath Perera氏がQuoraで非常にわかりやすく解説しています（https://oreil.ly/YkNwa）。複合イベント処理（CEP）は、イベントストリーム処理（ESP）のサブセットと考えられています（図6-2）。扱う問題やユースケースによって使い分けると良いでしょう。

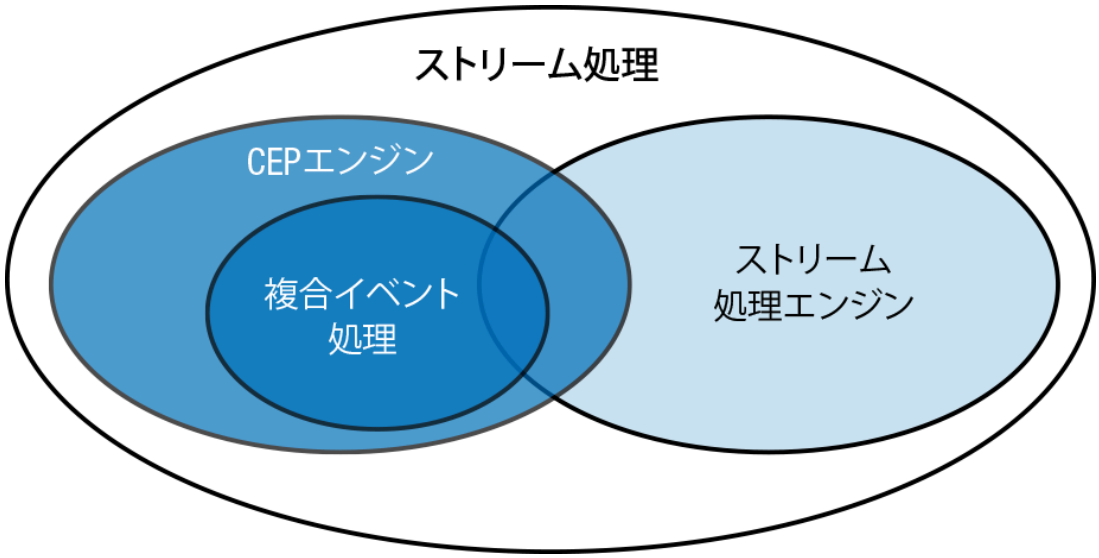

図6-2　複合イベント処理は、ストリーム処理のサブセット

時間順に並んだイベントを分析したり、処理グラフを作成したりする必要がある場合は、ESPを使ってください。しかし、より高度なクエリ言語を使ってイベントを並列に分析する必要がある場合は、CEPを使います。ストリーム処理エンジンは、データのクエリ（フィルタリング、順序の検証など）に重点を置きます。CEPエンジンは、タイムウィンドウの比較、異なるイベント間の比較や相関関係の検索など、複雑な分析に重点を置きます。

ただし、図の中央の重なりが示しているように、CEPの機能を採用するストリーム処理エンジンが増えているため、こうした違いは薄れ始めています。

また、ストリーミングプラットフォームの異なる使い方として、イベントに反応したりリッスンしたりするのではなく、アプリケーションの状態を転送するというシナリオもあります。状態転送の目的は、データを同期させることであり、イベントに対してアクションを起こすことではありません。この2つの目的は、組み合わせることもできます。例えば、イベントによる状態転送は、4章で説明したCQRSと組み合わされることがよくあります。

これらのさまざまな例から、どのような結論が得られるでしょうか。まず、イベントはアプリケーション間で通信される場合もあれば、1つのアプリケーションの内部で通信される場合もあります。したがって、ドメイン境界や再利用性などの問題を考慮する必要があります。次に、イベントは、分析目的にも運用目的にも使用できます。分析イベントや機械学習モデルによる予測は、業務上の意思決定を行うために、トランザクションシステムに戻される場合があります。そして、それぞれのユースケースには固有の特性があり、カスタムソリューションが必要になるケースがあります。そのため、アーキテクチャにコン

ポーネントを追加して補完し、そのガイダンスを作成する必要があるかもしれません。

では、少しズームアウトして、イベントプロデューサーとイベントコンシューマをどのようにすれば一番うまくいくのかを考えてみましょう。

6.2.6　イベントストリーミングプラットフォーム

イベントストリーミングプラットフォームは、イベント駆動型アーキテクチャのコンポーネントであり、ほかの多くの（サブ）コンポーネント（CEPエンジンやイベントブローカーなど）で構成することができます。イベントブローカーは、イベントプロバイダーとイベントコンシューマ間のさまざまな形式のやり取りを管理するため、不可欠です。イベントストリーミングプラットフォームのコアコンポーネントは、イベントやデータストリームの移動と処理に注力します。メッセージの受信や複数のコンシューマへのルーティングと送信などを行います。イベントストリーミングプラットフォームが、例えばAPIプラットフォームと異なる点は、大量のデータ（状態）を保存することができ、分散機能によってデータの損失を防ぐことができる点です。データのストリームが入ってくると、プラットフォームはそれらを変換、結合、フィルタリング、集約します。プラットフォームによっては、複雑なイベント処理機能も提供しているため、コンシューマ側やストリーミングプラットフォーム内で分析を行うこともできます。

このように、注意しなければならない複雑な点が数多くあります。この項では、イベント駆動型アーキテクチャが異なるドメイン同士を接続するた際に、内部で一般的に何が行われているのかについて説明します。簡単な例から始めて、次第に広げていきます。

6.2.6.1　EDA参照アーキテクチャ

プロデューサーとコンシューマの間で、ストリーム処理サービスを使ってイベントを単純に流通させる、軽量なアーキテクチャを提示します。図6-3は、イベントのストリームを受信し、機能（Function）を使用してそれらを処理し、下流で消費するために最終結果をほかのブローカーにルーティングする、イベント駆動型のサーバーレスアーキテクチャの設計例です。

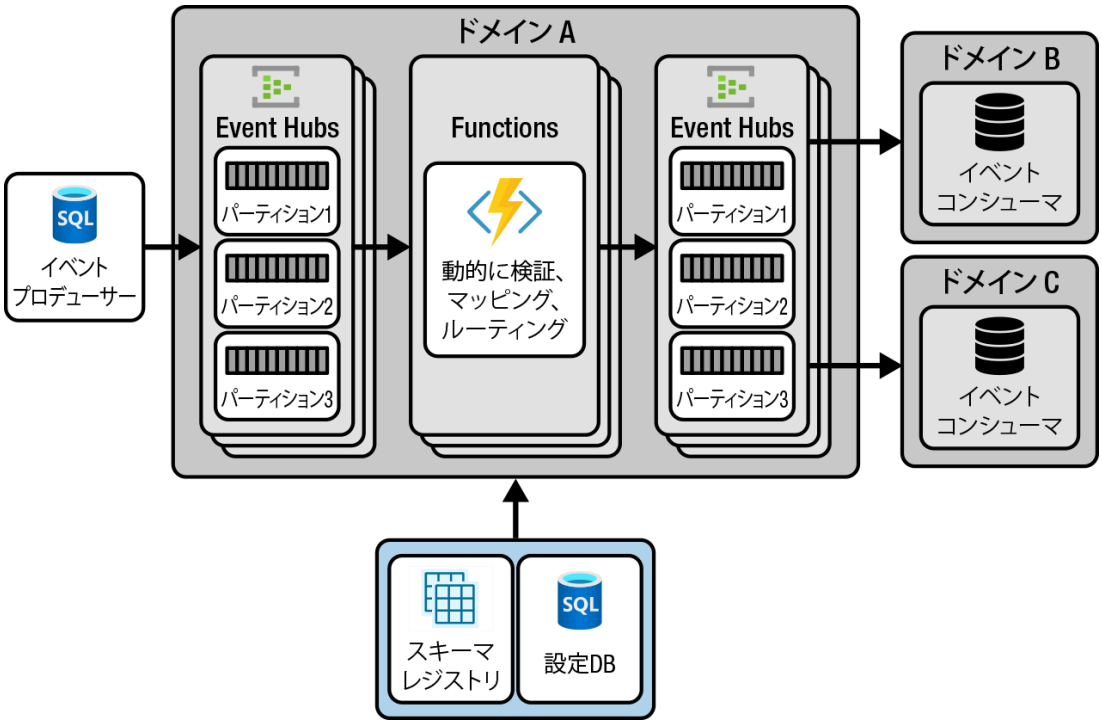

図6-3　Azure Event HubsとAzure Functionsを使用した、イベントプロデューサーとイベントコンシューマをつなぐ参照アーキテクチャ

この設計を見て、まず疑問に思うのは、イベントプロデューサーとイベントコンシューマの間に、なぜ2つのストリーム処理サービスと1つの機能があるのかということでしょう。まず、一連のイベントブローカーでプロバイダーとコンシューマを分離することをお勧めします。すべてのイベントプロデューサーが安定した配信とユーザーフレンドリーなイベントの配信を提供できるとは限らないからです。Webアプリから直接メッセージを取り込む状況を想像してみてください。このような状況では、ドメインがアプリケーション設計に破壊的な変更を加えてしまう場合があるかもしれません。そのようなドメインは、イベントを元の構造にマッピングして安定した消費を実現したり、同じプロダクトの異なるバージョンを呼び出す方法を提供したりする必要があります。ストリーム処理サービスを追加することで、ドメインは、既存のすべてのコンシューマとの互換性を検証することなく、進化し、破壊的な変更を加えることができます。さらに、イベントがコンシューマに提供される場合に、あまりにも生だったり技術的な形式だったりするかもしれません。レイヤーを追加することで、プロバイダーをコンシューマから分離できます。また、イベントブローカーを追加することで、コンシューマが複数の地域に分散するような将来のシナリオも実現できます。すべてのコンシューマがイベントを集めるためにネットワークを渡り歩くのではなく、各地域にイベントブローカーを置くことで、コンシューマにデータを近づけることができます。

Kafkaを使用し、変換サービスを必要とするアーキテクチャについては、Decodable（https://www.decodable.co）やksqlDB（https://oreil.ly/GRzsU）にアクセスしてみてください。これらは、Kafkaパイプラインでデータを変換するためのSQLインタフェースを提供します。

図6-3では、イベントの受信、検査、ルーティングを行う機能が示されています。これらの機能は状態を保持しません。このパターンの利点は、検証を追加できることです。例えば、イベント本体を検証し、重複している可能性のあるものを検出したいと思うかもしれません（まだ説明していないコンポーネントを使用します）。さらに、プロデューサーが破壊的な変更を行った場合に、読み出しやすくするためにメッセージを再構築したり、元の形式にマッピングしたりしたくなるかもしれません。機能を使えば、インフラストラクチャーを追加することなく、このようなことが実現できます。

アーキテクチャをさらにセルフサービスにするには、ルーティングとマッピングをメタデータ駆動型にする設計が考えられます。例えば、設定ストアを追加して、どのイベントストリームがほかのどのイベントストリームに関連するかを示すメタデータを保持するように追加します。そして、機能はイベント処理中にこのストアから読み込みます。この方法を発展させ、コンシューマが見ることができるメッセージや属性を決めるために、小さなコントロールフレームワークを実装することもできます。さらに、ガバナンスのためにスキーマレジストリ（https://oreil.ly/01hPO）をアーキテクチャに追加することもできます。このようなレジストリによって、技術的にメッセージを検証したり、ドメイン境界などの一定のビジネス基準に基づいて、類似したスキーマを論理的にグループ化したりすることができます。

6.2.6.2　データプロダクトの作成

最初の参照アーキテクチャを基に、イベントを永続化できるようにして、バッチデータ処理と（後で）結合できるようにしましょう。図6-4では、データを

永続化するために、データプロダクトアーキテクチャを追加しています。リアルタイムのデータインジェストを実現し、データプロダクトを作成します。

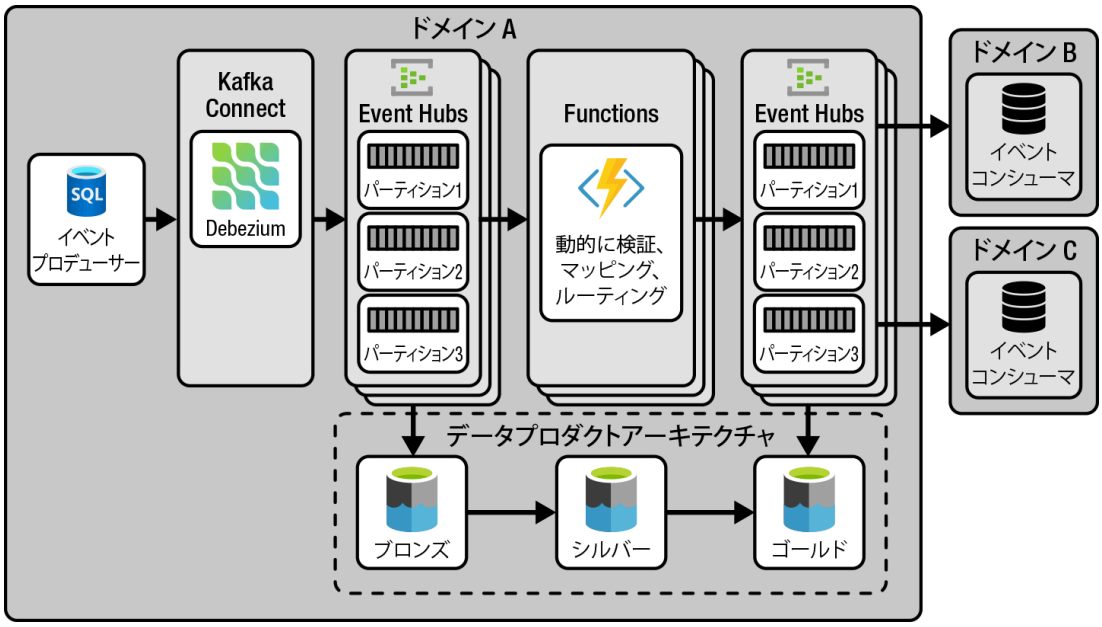

図6-4　データプロダクトアーキテクチャを追加し、イベントの永続化を実現

この設計を使って実装できるパターンは2つあります。1つ目は、インジェストのときにイベントを直接取り込む方法です。このパターンは、異なる速度で到着するほかのデータとイベントを統合したい場合や、アドホックな分析を行う予定がある場合、または将来の分析のためにこのデータをデバッグしたい場合に便利です。このような状況では、イベントは、提供アプリケーションの内部アーキテクチャと整合します。下流のコンシューマにとっては、構造が生過ぎたり技術的過ぎたりする可能性が高いので、このデータをほかのドメインに直接配布することを避けるという原則を策定したくなるでしょう。

話をシンプルにするために、次の説明ではデータプロダクトアーキテクチャを削除します。ただし、実際には、最新型の総合アーキテクチャには、この章で説明したすべての特徴が組み合わされています。

リアルタイムイベントデータを履歴化するための第2の方法は、消費レイヤーを利用することです。論理的には、この時点のデータは、ほかのドメインが下

流で消費するのに使いやすい形にモデル化されています。ストリーミングをデータプロダクトアーキテクチャの消費レイヤーと組み合わせることで、ドメインはデータプロダクトの変化をより迅速に知ることができます。このため、（バッチのように）毎日ではなく、例えば1時間ごとや数十分ごとにドメインがデータプロダクトを更新できるようになります[†8]。

[†8] Joe Plumb氏は、SQL、Azure Functions、Data Factory、Event Grid を用いてデータパイプラインを作成し、1分に1回データを処理する方法について、ブログ記事（https://oreil.ly/Xn51N）を書いています。

6.2.6.3　イベントストア

次の段階では、図6-5に示すように、イベントストアをアーキテクチャに追加します。**イベントストア**は、イベント駆動型アーキテクチャにおいて、イベントを保存するデータベースにほかなりません。このコンポーネントは、検索や複雑なクエリを実行する際に役立ちます。例えば、メッセージをほかのデータ（参照データなど）でリッチ化しなければならない状況を想像してみてください。イベントストアは、このようなデータを取得するのに最適な場所となります。

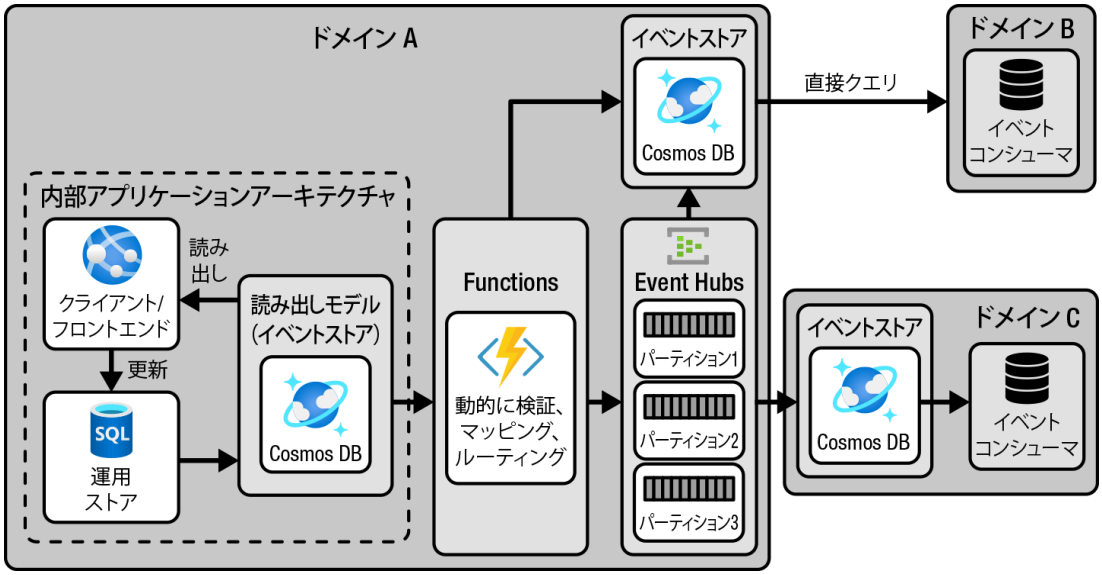

図6-5　イベントストアをアーキテクチャに追加

イベントストアは、データをクエリする際の柔軟性を提供し、情報の長期保存を実現します。イベントストアは、データを提供する側で、読み出しストアや

コマンドストアの役割を果たすことも、データを消費する側で、クエリ可能なデータベースの役割を果たすこともできます。

イベントストア内のメッセージは、消費後に削除されることはありません。実際、イベントストアは単なるメッセージのリストであり、いつでも最初から読むことができます。そのため、どのコンシューマも次に処理するアイテムを自由に決めることができます。この機構により、複数のコンシューマがそれぞれのペースでメッセージを処理することが可能になります。特定のコンシューマが特定のメッセージを最初からやり直したり、ある特定のメッセージを再処理したい場合も、イベントストアのおかげでそれが実現できます。イベントを再処理するときに、イベントを修復、操作、リッチ化することができます。例えば、エラーを発見した場合、過去にさかのぼって修正を適用し、すべてのイベントを新しいイベントとして再実行することができます。この巻き戻しパターンは、さまざまなイベントストリームを組み合わせるためにも使用できます。

イベントストアに適しているのは、**図6-5**に示すように、Azure Cosmos DB（https://oreil.ly/KRO4K）のようなNoSQLデータベースです。NoSQLデータベース、特にドキュメントストアは、非構造化データの取り扱い、処理、保存方法において柔軟性があります。NoSQLデータベースはスキーマレスであり、データベースに入力されるデータに対してデータ構造が指定されることはありません。関係データベースを使用することもできますが、その場合、データがデータベースに格納される前に、前もって関係データモデルを定義する必要があります。また、必要に応じてイベントストアを使い分けることもできます。例えば、アーキテクチャに関係データベース、NoSQLデータベース、グラフデータベースを組み込んで、多様なニーズに対応することができます。

イベントストアをアーキテクチャのデータ生成側に配置すると、ドメインの内部アプリケーションアーキテクチャに近い場所でデータを保持することができます。例えば、イベントストアをCQRSの読み出しストアとして使用するなど、イベントストアを運用アーキテクチャの一部にすることもできます。この場合、イベントストアは読み出しレプリカの役割を果たし、データをより高速に読み出せるように効率化されます。つまり、運用データベースからデータを読み出す代わりに、イベントストアからデータを読み出すことになります。このようなアプリケーション設計は、アプリケーションをより高速で柔軟なものにします。例えば、読み出しストアとコマンドストアの両方を同時に拡張するのではなく、それぞれを個別に拡張することができます。

また、運用アーキテクチャの中で、イベントストアをコマンドストアとして使用することもできます。これにより、**コマンドソーシング**が可能になり、コマンドが処理される前に、まずそのコマンドを永続化することができます。これにより、システムの全状態を再現することができます。メッセージを監査し、何が起こったかの証跡を確認し、処理される前にコマンドをコントロールすることができます。

アーキテクチャのデータ消費側にイベントストアを配置すると、コンシューマは運用アプリケーションにコールバックすることなく、別ドメインのデータを直接やり取りできるようになります。例えば、コンシューマは複雑なクエリをネットワーク経由で送信することなく実行できます。また、ドメインがそれぞれのニーズに合わせてイベントを変換し、自身のイベントストアに保存することもできます。

Cosmos DBのような最新のNoSQLデータベースが提供する機能に、**変更フィード**（https://oreil.ly/blJmK）という重要な機能があります。この機能は、イベント駆動型アーキテクチャを支える重要な機能であり、より多くのデザインパターンを実現可能にします。例えば、データがデータベースに格納されるたびに機能をトリガーすることができます。つまり、データベースから新しいデータを読み出すためにワーカープロセスを使うのではなく、データベースにデータが入ったときにその場で検証や解析を行うことができます。この仕組みは、Apache KafkaでksqlDBを使うのと似ています。

クラウドネイティブのNoSQLデータベースをイベントストアとして使用することには、長所と短所があります。長所としては、例えば、反応型の機能によってアーキテクチャが強化されます。イベントをより簡単に処理できるようになり、クエリ可能なイベントストアを持つことができます。これは、アーキテクチャ全体をシンプルに保つ上で重要な利点です。一方で、このような高性能なデータベースは高価になりがちです。これは多くのユースケースにとって問題となります。

クラウドネイティブのNoSQLデータベースを、例えばKafkaで置き換えることは可能でしょうか。可能ですが、Kafkaはメッセージを永遠に保存するためのものではありません。Kafkaは、Azure Event Hubsのように、データインジェストのスループットが高く、メッセージを自分のペースで消費したい複数のコンシューマグループに配信する必要があるユースケースに適したものです。

次の項では、より複雑なユースケースを見ていきます。このユースケースでは、高速なデータインジェストと分析機能が必要となります。

6.2.6.4　ストリーミング分析

分析ユースケース（複合イベント処理）をサポートするには、アーキテクチャに新たなコンポーネントを追加しなければなりません。ユースケースの要件に応じて、リアルタイム分析、視覚化などのコンポーネントが必要になります。これらのコンポーネントの中には、大量のデータを処理できるように、分散処理アーキテクチャを持つものもあります。以下にいくつかシナリオを示します[†9]。

[†9] Azure Samples（https://oreil.ly/Jz6a7）では、大規模ストリーミングを実現するためのソリューション例をいくつか紹介しています。

- リアルタイムダッシュボード機能を提供するために、イベントを分析し、レポートデータセットに直接プッシュします。実際の例としては、4時間以内にさまざまな地域からログインを試みた従業員を一覧にする、リアルタイムのレポーティングが挙げられます。
- イベントを分析し、データを補完します。例えば、それぞれのイベントをまず不正検出モデルにルーティングし、イベントにスコア付けを行い、次にこれらのイベントをイベント駆動型アーキテクチャに戻して下流で使用するようにします。
- 別のデータベースから情報を得て、イベントを分析します。例えば、顧客が新しいクレジットカードの発行を請求した場合、提供された身分証明書がまだ有効かどうかを調べるなどです。
- 何百万ものセンサーデータを時系列データベースで処理する場合、その前にまずメッセージをクレンジングし、正しくタイムスタンプを付与したいでしょう。
- 地理的な情報を使ってトラフィックデータを処理します。そのために、すべてのイベントをDatabricksのような高性能処理プラットフォームにプッシュします。

設計を変更して、ストリーミング分析を実現するためのコンポーネントを追加したものは、図6-6のようになります。ストリーミングサービスは、イベント

プロデューサーとイベントコンシューマの双方に大きな価値をもたらします。可能性は無限です。

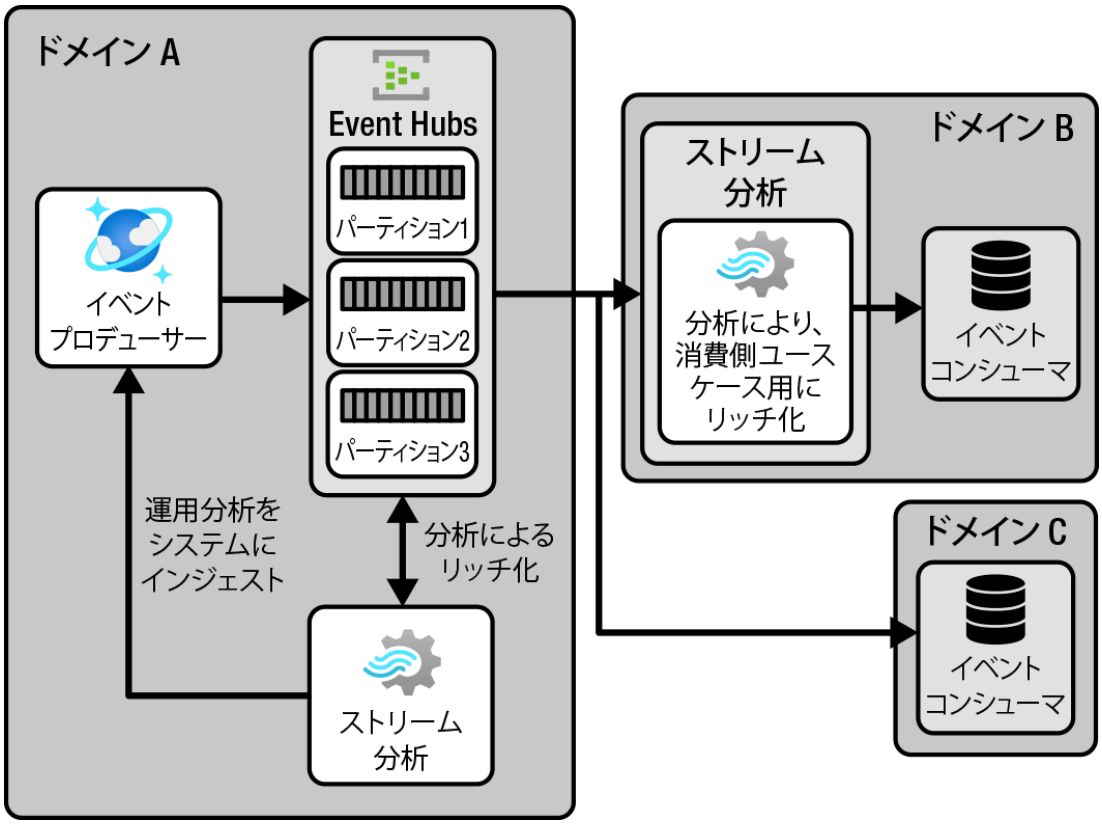

図6-6　ストリーミング分析コンポーネントを追加した参照アーキテクチャ

アーキテクチャを適切に管理するには、サービスやプラットフォームをドメインに依存しないようにすることをお勧めします。すべての分析コンポーネントとそれに付随するビジネスロジックを、アーキテクチャの提供側または消費側のいずれかに配置しなければなりません。分析サービスを追加してアーキテクチャを補完することもできますが、それは汎用的なデータ品質プロファイリングサービスなど、ドメインに関係しない機能だけに限定しましょう。

ここまでのイベント駆動型アーキテクチャに関する議論から、どのような結論が得られるでしょうか。まず、管理しなければならないものがたくさんあります。イベントを流通させるためのイベントブローカー、データをクエリしたりキューに入れたりするためのイベントストア、データプロダクトのためのインジェストパイプライン、データを価値に変えるための分析コンポーネントなどです。これらのコンポーネントをアーキテクチャに組み込む場合、ドメインにこれらのコンポーネントすべてに関するガイダンスを提供する必要があります。次に、境界には2種類あります。アプリケーションコンポーネント間のアプリケーション内イベントストリームと、異なるドメインのアプリケーション間の通信です。どちらも、ガバナンスが必要です。

6.2.7　ガバナンスモデル

さて、イベントプロバイダーとイベントコンシューマのガバナンスモデルを見てみましょう。イベント駆動型アーキテクチャは、データプロダクトやAPIをほかのドメインに提供する際のモデルと同じモデルに従うべきです。イベントストリームやキューなどの一連のイベントは、すべてイベントプロバイダー（データプロバイダー）が所有しなければなりません。ドメイン間でサービスを共有し、それらをアプリケーション内部とドメイン間の通信の両方に使用する場合、ほかのドメインのコンシューマには内部の通信チャネルを覗かれたくないため、すぐに問題が生じます。これを避けるために、アプリケーション内部イベントストリームとドメイン間イベントストリームをしっかりと分離することをお勧めします。このことは、ローカルストレージを使用するデータプロダクトアーキテクチャへのデータインジェストについて説明したときのことを思い出してください。

アプリケーション内部とドメイン間のイベントストリームは、さまざまな方法で分離できます。ベストプラクティスは、複数のサービスを介して分離を行うことです。例えば、ランディングゾーンのサービスはドメインには関係ないものにして、ドメイン間配信のためのものにします。一方で、運用アカウントやサブスクリプションのサービスはアプリケーションの内部アーキテクチャのためのものにします。別の方法として、例えばデータ管理のランディングゾーンに専用のプラットフォームを配備して共有することもできます。この場合、分離の方法は異なります。専用のイベントストリームやイベントハブを使用するのも1つの方法です。命名規則を使って分離することもできます（ドメイン名を埋め込んだり、接頭辞を使用するなどしてキューやイベントストリームの名前の一部にします）。さらに厳密な方法は、コンシューマグループや、証明書、複数のクラスタを使用してイベントストリームを分離するという方法もあります。

（2つのアプリケーションコンポーネント間などの）内部通信に使用されるイベントは、技術的なものであってもかまいません。これらのイベントのストリームは**内部ストリーム**と呼ばれ、アプリケーションまたはドメインロジックの一部とみなされます。つまり、内部ストリームに入るイベントは、ビジネスストリームの入力として使うことができますが、生成アプリケーションドメインによってのみ消費され、そのほかのドメインによって直接消費されることはありません。これらのストリームのメタデータポリシーも、それほど厳密でなく

てかまいません。例えば、内部ストリームは、キューやイベントストリームのレジストリに、あまり詳しく説明しなくてもかまいません[†10]。

[†10] **レジストリ**は、オーナーシップ、バージョン管理、スキーマメタデータ、ライフサイクル管理など、すべてのキューとイベントストリームを登録するための中央の格納場所です。

対照的に、ドメイン間ストリームは、ほかのドメインによる消費だけが可能です。これらのストリームは**ビジネスストリーム**とも呼ばれ、ビジネスで消費できるようになっているため、読み出しに適した状態になっています。これらのストリームは分離されており、以前のバージョンとの互換性が保たれ、データコンシューマ向けに内部ストリームよりも詳しくレジストリに登録されています。これらは、2章のデータ管理の原則にすべて従っている必要があります。

次の項では、イベント駆動型アーキテクチャに移行するためのベストプラクティスをいくつか紹介します。

6.2.8　データプロダクトストアとしてのイベントストア

データを長期保存する場合、ストリーミングプラットフォームをデータベースとして使用し、データプロダクトストアのような役割を担わせることはできるのでしょうか。もちろんできます。イベントソーシングや、状態ストア、最新のストリーミングプラットフォームの巻き戻し機能により、最新のデータが継続的に格納される、連合型データプロダクトを作成することができます。コンシューマは、巻き戻し機能を使って、データのコピーを生成できます。巻き戻した後は、ストリーミングを使用して、リアルタイムに、データのコピーを最新状態に保つことができます。この方法は、アプリケーションのトランザクションログからすべての変更を読み出し、変更データキャプチャ（CDC）を使って実現できます。

データを長く保持する場合、どのような条件であれば、ストリームが長くまたは無限（永久）にデータを保持できるかを判断すべきです。監視を行わずに保持期間を長くすると、アーキテクチャが高コストになるかもしれません。また、GDPRなどの保有ポリシーやデータライフサイクル管理ポリシーに違反する可能性もあります。そのため、何らかの管理が必要です。原則としては、最

新または一意のメッセージのみを保持するように設定することです。別の方法として、より安価なストレージを使用して、データプロダクトアーキテクチャにメッセージのオフロードをすることもできます。

6.2.9　アプリケーションバックエンドとしてのイベントストア

中央ストリーミングプラットフォームは、アプリケーションのバックエンドとして使用できるのでしょうか。例えば、Kafkaをアプリケーションのデータストアとして使うことはできるのでしょうか。理論的には可能ですが、一般的にはお勧めしません。

これにはいくつかの理由があります。まず、ストリーミングの性質上、非同期であるという点です。遅延に関して要件のないアプリケーションでは問題ないでしょう。それでも保持制限をなくす必要があります。保持制限があると、アプリケーションデータが失われるリスクがあります。また、アプリケーションと、エンタープライズデータ流通の目的が密結合になってしまいます。これらは2つの異なる関心事ですので、そのガバナンスを混在させるべきではありません。

ストリーミングプラットフォームをアプリケーションのバックエンドとして利用すると決めた場合、このような状況にどのように対処するか、明確なガバナンスを構築するようにしてください。ドメインごとにこのパターンのスコープを明確にするのも良いでしょう。また、特定のドメイン用に別のプラットフォームを立ち上げるという方法もあります。ストリーミングプラットフォームをバックエンドとして使用する際の最大のリスクは、統合データベース（https://oreil.ly/SHDrh）を作成してしまうことです。

ほかのストリームから派生したストリームについても、同じ原則が適用されます。これらのイベントストリームをほかのコンシューマが直接消費できるようにすると、イベントストリームの上にイベントストリームが重なり、行ったり来たりする無秩序な状態になる可能性があります。このようなスパゲティ構造になってしまうのは、プラットフォームの利用において、チームを統括するコントロールとポリシーが欠如しているためです。原則としては、コンシューマがイベントをストリーミングプラットフォームに戻す前に、まず自分のアプリケーションにインジェストするように強制してください。

6.3　バックボーンとしてのストリーミング

1章では、トランザクションシステムと分析システムが密接に連携していくトレンドについて説明しました。イベントストリーミングプラットフォームは、この2つのシステムを接続するソリューションです。このコンポーネントは以下を実現可能にします。

- 運用システムを拡張して、すべての過去データを記録するインメモリシステムを追加します。運用システムで、意思決定のために過去コンテキストが必要となるときは、ストリーミングプラットフォームがその過去データを提供します。
- リアルタイムな意思決定ロジックで運用システムを拡張します。例えば、顧客が新しい商品を購入したときに、財務システムに照会して未払い残高を確認します。ストリーミングは、この処理を円滑に行えるようにします。
- コマンドソーシングを利用してコマンドを永続化し、システム全体の状態を再現します。これによって、例えば、記録し損ねていた注文をリストに追加して、後から処理を行うことができるようになります。

大規模な非同期メッセージングシステムを構築して運用する場合、一時的に負荷が高まり、システムが処理可能な頻度よりも高い頻度でデータを受信する場合があります。この問題は**バックプレッシャー**（https://oreil.ly/8wbF5）と呼ばれるものです。正しく対処しないとリソースの枯渇や、最悪の場合、データ消失につながるリスクがあります。この問題に対処するには、さまざまな方法があります。リソースを追加するのも1つの解決策です。また、キューに制限を設け、制限を超えたメッセージは捨てたり、別の場所に保存したりする解決策もあります。

同じアーキテクチャを使って、運用システムを、複数のテクノロジーで構成されるバックエンドを持つように設計できます。ここでのストリーミングは、さまざまなデータベース間でデータを同期させたり、データの結合や統合をサポートしたりします。マイクロサービスベースのアーキテクチャでは、イベント

駆動型アーキテクチャも使うことが多いです。イベントストリーミングは、マイクロサービス同士の通信を実現するパターンの1つです。

6.4　保証と一貫性

ストリーム処理は、バッチ処理よりも複雑です。バッチ処理で何か問題が発生した場合は、すべてのデータを再送信し、バッチ処理を再実行することで問題を解決することができます。ストリーム処理の場合に、障害や不整合へ対処するには、いくつか考慮しなければならないことがあります。

6.4.1　一貫性（整合性）レベル

まず、考慮すべきなのは、アプリケーションで必要とする一貫性（データの完全性と正確性）のレベルです。ストリーミングには、2種類の一貫性があります。

結果整合性
このモデルは、すべてのデータが最終的にプラットフォームに反映されることを保証するものです。結果整合性は保証としては弱いのですが、重要ではないデータには有効です。

強一貫性
このモデルは、すべてのイベントが順番に処理されることを保証します。プラットフォームは、すべてのデータのコピーに対して更新情報を必ず伝搬します。エンティティやノードのいずれかが更新されると、すべての複製が更新されるまでロックされます。

強一貫性を実現するのは難しいかもしれません。イベントの速度が非常に速く、ストリーミングプラットフォームが追いつくのがやっとの場合、すべてのメッセージを正しく受信するように設計するのは難しいか不可能になることがあります。そのため、強一貫性を必要とする設計は、さまざまなユースケースでは受け入れられず、結果整合性を使わざるを得なくなる場合もあります。このようなケースでは、正しいことを保証するために処理を追加しなければならないかもしれません。例えば、データベースを追加してすべてのデータを永続化し、定期的に照合を行い、すべてのデータが正しく同期されたことを確認するという方法があります。また、CDCのようなツールを追加して、すべての変更が正しく配信されていることを確認したり、新たにサービスを構築して、欠落したメッセージを読み出し、検証し、再送信することによって、一貫性を確保したりすることもできます。

6.4.2　処理方法

モデルと強一貫性モデルには、さまざまな処理方法が存在します。**少なくとも1回処理**は、メッセージが失われないことを保証します。ただし、すべてのメッセージが一意であることや、一度だけ配信されることを保証するものではありません。イベントが重複することや、イベントが複数回処理されることが許されます。

正確に1回処理は、各イベントが正確に一度だけ配信されることを保証します。**1回のみ配信**モデルとも呼ばれます。メッセージが失われることもなく、イベントが重複することもありません。このモデルを実装するには、再配信パターンを追加する必要があるかもしれません。識別子を追加して、各メッセージの一意性を検証する場合もあります。

最後に紹介するモデルは、**多くても1回処理**です。**無保証処理**と呼ばれる場合もあります。このモデルでは、すべてのメッセージが正しく配送されることは保証されません。メッセージは失われてもよく、また、何度も配送される場合があります。

ユースケースの要件やそのほかの制約によっては、異なるモデルを組み合わせて使用することができます。例えば、結果整合性モデルでは、最初は受信を確認せずにメッセージを配信し、時おりすべてのメッセージを比較し、重複を削除し、欠落したメッセージを再配信して、正確に一度だけ配信されることを保証できます。

6.4.3　メッセージ順序

また、一般的に、ストリーミングで問題となるものに、メッセージ順序があります。例えば、場合によっては、すべてのメッセージを厳密に生成順にしなければならないかもしれません。Kafkaではこれを実現することができますが、1つのパーティションを使っている場合だけです。スケーラビリティのために複数のパーティションが必要な場合は、`row_no`など、メッセージ内にプロパティを追加して、このプロパティを使って順序付けを行う必要があります。この場合、配信側が配信の際に何らかの調整を行う必要があるかもしれません。または、すべてのメッセージをコンシューマ側で消費した上で、コンシューマ側で並べ替える方法もあります。

6.4.4　デッドレターキュー

ストリーミングプラットフォームの中には、**デッドレターキュー**と呼ばれる、正常に配信されなかったメッセージのためのキューが追加されているものもあります。正常に配信されなかったメッセージとは、サイズ制限を超えたメッセージや、不正な形式のメッセージ、存在しないイベントストリームに送信されたメッセージ、設定された時間内に処理されなかったメッセージなどです。このような場合、メッセージはシステムから削除されず、デッドレターキューに入れられます。デッドレターキューにメッセ時が入った場合は、通常、開発者は何が問題だったのかを手作業で調べる必要があります。多くの場合、イベント提供側のアプリケーションの変更が十分にテストされていないのが理由です。このような場合、問題のあるメッセージを移動するか、元のキューに再送信する必要があります[†11]。

[†11] このトピックについては、Ning Xia氏のブログ記事“Building Reliable Reprocessing and Dead Letter Queues with Apache Kafka”（https://oreil.ly/-4Z0J）を参照してください。

6.4.5　ストリーミングの相互運用性

パブリッシャーとサブスクライバー間のインタフェースについては、テクノロジー面での選択と、設計面での選択を行う必要があります。プラットフォームによっては、提供側アプリケーションがどのようにプラットフォームと接続しなければならないか、どのようなプロトコルを使用しなければならないかを規定しているものもあります。オープンで幅広いメッセージプロトコルをサポートし、複数のプログラミング言語間で透過的な相互運用を可能にするプラットフォームもあります。中間的な方法として、インタフェース記述言語（IDL）（https://oreil.ly/2CLeH）を使用する方法があります。IDLを活用すれば、メッセージをエンコードすることで、データ形式や型の検証を行えるようになります。一般的には、このようなプロトコルを使います。

また、データシリアライゼーションをサポートするフレームワークも検討すべきでしょう。データシリアライゼーションフレームワークは、インタフェース間でデータを移動する際の互換性や、バージョン管理、ポータビリティにおいて重要な役割を果たします。データシリアライゼーションフレームワークは、データを標準形式へと一貫性のある変換を行ってくれます。変換後、多くのシステムや、プログラミング言語、ほかのフレームワークからデータを使用でき

るようになります。また、これらのシリアライゼーションフレームワークは、圧縮やセキュリティ（暗号化）などの機能により、データ処理を改善してくれます。このため、シリアライゼーションは、データプロダクトアーキテクチャにデータを流通するのに適しています。シリアライゼーションフレームワークやプロトコルとしては、Apache Avro、Protocol Buffers、Apache Thrift、MQTT、AMQPなどが人気があります。

ストリーミングプラットフォームからメッセージを提供・消費する方法としては、JSONやXMLメッセージをサポートするRESTful APIを使用する方法もあります[†12]。オンラインチャネルやWeb環境に適した方法です。JSONはスキーマレスで柔軟性が高いという利点がありますが、1つ大きな欠点があります。JSONはスキーマエンコーディングを強制しないので、互換性が保証されません。つまり、メッセージの柔軟性と強制力はトレードオフの関係にあるのです。

[†12] Kafkaで、この形式の通信を行うには、REST Proxy（https://oreil.ly/vkjNc）をインストールしてください。

6.5　ガバナンスとセルフサービス

イベント駆動型アーキテクチャには、メタデータ機能が付加されています。その中でも重要なのは、イベントストリームレジストリ（スキーマレジストリ）です。次のような目的で使われます。

オーナーシップ登録
イベントは必ず所有されます。このレジストリは、イベントのオーナーシップを管理します。また、メッセージキューとイベントストリームを内部用なのかドメイン間用なのかに分類するのにも使えます。

スキーマドキュメント管理
すべてのメッセージのスキーマの構成とドキュメントを管理することをお勧めします。XMLやJSONの場合、これらはXMLスキーマやJSONスキーマとなるのが自然です。Avroなどのシリアライズフォーマットは、独自のIDLを使用します。イベントをドキュメント化して管理する方法として、AsyncAPI（https://www.asyncapi.com）をお勧めします。AsyncAPIは、イベント駆動型アーキテクチャを改善するオープンソースの取り組みです。すべてのスキーマとドキュメントを一緒に管理するには、レジストリのようなツールを使用する方法が良いでしょう。

バージョン管理

スキーマはバージョン管理されていなければなりません。もしバージョン管理されていない場合、何か変更を加えると、イベントコンシューマに直接影響を与えてしまうかもしれません。スキーマのバージョンを管理するためには、レジストリが適しています。

リネージ

残念ながら、イベントのリネージは複雑です。また、使用するストリーム処理エンジンに依存します。状況によっては、独自のコードや実装が必要になるかもしれません。例えば、カタログやデータガバナンスソリューションの一部としてAPIを呼び出すことなどです。

セルフサービスによって運用モデルを改善する場合は、EDAのメタデータが欠かせないものになります。例えば、ドメインがオフセットをリセットしたり、セルフサービスでキューやトピックをプロビジョニングしたりしたい場合があります。そして、このような要求を自動的に承認されるようにしたい場合もあるでしょう。また、RESTプロキシのテンプレートをドメインで設計し、適切なセキュリティポリシーで自動的に配備したい場合もあります。さらに、ドメインがコンシューマにメッセージを渡す前に、データのフィルタリングを適用することもできます。筆者のお勧めは、セルフサービスを念頭に置きながら、これらのパターンを1つずつ利用できるようにしていくことです。

6.6　まとめ

さて、イベントに関する内容が盛りだくさんでした。イベントが持つパワーをご理解いただけたと思いますが、同時にイベントの複雑さもご理解いただけたと思います。イベントによって、アプリケーションを接続し、データを連合し、プロセスをデジタル化し、社内外のコミュニケーションを改善することができます。イベントによって、システムが互いにどのように反応するかを理解することができ、日々の業務改善に役立ちます。

これまで見てきたように、ストリーミング処理は複雑です。継続的にデータを取得し、データストリームをプラットフォームにインジェストするには、アーキテクチャに多くのコンポーネントを追加する必要があります。このような複雑な多様性を考慮すると、例えば、正確に1回処理を保証するためには、さまざまな多くのツールセット、フレームワーク、サービスを追加して使用することになります。筆者がお勧めするのは、徐々に構築し、基本ルールをいくつか設定することです。

- ドメインチームに提供するサービスやプラットフォームを標準化しましょう。ツールやサービスの中には、より特化したものもあれば、できるだけ包括的なものもあります。上手くバランスをとりましょう。相互に排他的かつ全体的に網羅的なものが良いアーキテクチャです。サービスが提供する機能はあまり重ならないようにした上で網羅的なものにします。
- イベント処理の標準を設定します。イベントは、すでに起こったことを表すものなので、その定義からわかるように不変です。
- イベントプロデューサーにデータインジェストの責任を負わせます。イベントプロデューサーの役割は、「少なくとも1回」の配信を保証することです。一方、コンシューマは「多くても1回」の配信に対応します。
- イベントストリームを分離します。それぞれのサービスは、ある特定のドメインからのイベントストリームを所有しなければなりません。
- ドメイン境界を越えてデータを移動する際は、結果整合性を推奨します。
- ビジネスロジックに注意しましょう。ドメイン間通信のためのプラットフォームやサービスは、できるだけ独立したものにしてください。
- 内部アプリケーションとドメイン間流通を明確に分離するようにします。境界を越えるときは、前の章の原則をすべて適用してください。例えば、境界を越える場合、メッセージはそのドメインのユビキタス言語に従うべきです。
- 最初から自動化を意識しましょう。イベント流通が成功した場合、数百から数千のイベントストリームを管理しなければならなくなります。アーキテクチャをスケーラブルにするには、すべてのイベントにスキーマを定義し、契約ファーストの考え方を採用します。
- イベント、通知、クエリ、コマンドを分離します。それぞれについて原則を策定するのも良いでしょう。例えば、クエリはシステムの状態を変更しないこと、コマンドは何らかのデータを作成、更新、削除すること、などです。コマンドは明示的にトランザクション境界内になければなりません。コマンド処理を複数のアプリケーションに分散させた場合の最大のリスクは、一貫性やトランザクションの整合性が保証されなくなることです。さらに、コマンドが拒絶されて、デッドレターキューに入てしまう場合もあります。本来であれば、コマンドが拒絶された場合、拒絶されたことを通知してほしいはずです。
- エンジニアの複雑な作業を簡単にするために、サービスの上にラッパーを構築することを検討してください。

これらの原則を検討する際、必ず考慮しなければならないことがあります。イベントベースのアーキテクチャは複雑ですので、組織に大きな価値をもたらす

場合にのみ意味があるということです。スピードが求められていないところに、イベント駆動型通信が使われている場面を数多く経験してきました。このため、組織として、ストリーミングを使うことが適切であるかどうかを検証する原則を設定すべきでしょう。複雑さが増しても仕方がないと思えるようなビジネスバリューが必要です。

この章では、アプリケーションの接続とアプリケーション間のデータ流通について多くを学びました。次の章では、ヘリコプターに戻り、一段高いところから、データ流通とアプリケーション統合のすべてのパターンを俯瞰します。まずは、これまでの3つの章を簡単に振り返り、さまざまな側面を結びつけていきます。その後、変化に対応可能なエンタープライズアーキテクチャについて考えていきます。テーマが次々と切り替わるのでご注意ください。そのため、乱気流が発生する可能性があります。安全な空の旅をお楽しみください。